UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
FACULTAD DE INGENIERIA EN SISTEMAS



PROYECTO FINAL: PIPELINE CI/CD CON ASEGURAMIENTO DE CALIDAD INTEGRADO

ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE (SQA)
NOVIEMBRE DE 2026

CARLOS ALFREDO SANDOVAL SAGUI
CARNÉ: 7690 22 2568

1. Objetivo y alcance

Este proyecto responde al curso de Aseguramiento de la Calidad del Software. Tomé una aplicación que ya existía, el sistema de control del Transmetro (líneas, estaciones, buses, pilotos, alertas y reportes), y le armé un pipeline de integración continua para que cada cambio pase por verificaciones automáticas antes de entrar a la rama principal.

El repositorio está en https://github.com/SANDO-07/transmetro y es público, así que se puede revisar sin invitación.

1.1 Objetivo general

Implementar un pipeline CI/CD con pruebas automatizadas, análisis estático, medición de cobertura y quality gates, y medir cuánto mejora la calidad del proyecto entre el estado original y el final.

1.2 Alcance

1. Pruebas unitarias, de integración y de regresión con PHPUnit (58 en total).
2. Cobertura de líneas con PCOV y un umbral mínimo de 60 %.
3. Análisis estático con Laravel Pint, Larastan (PHPStan nivel 5) y SonarQube Cloud.
4. Pipeline en GitHub Actions con cuatro etapas: build, test, analyze y report.
5. Rama main protegida: los cambios entran solo por pull request con cinco checks obligatorios.
6. Documentación en la carpeta docs: Plan SQA, Matriz de Pruebas, Línea Base, Reporte de Mejora y Manual Técnico.

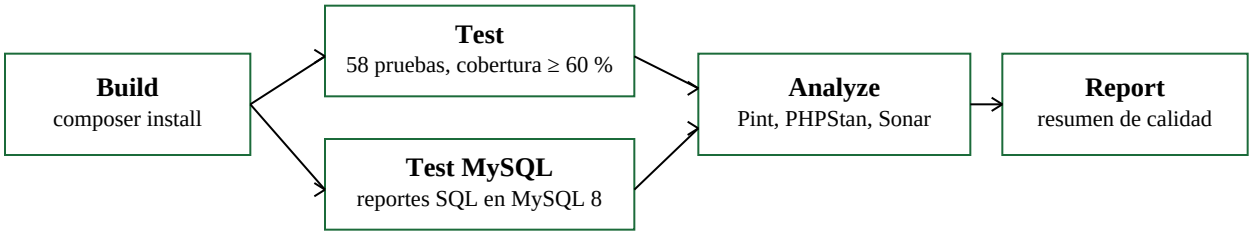
2. Stack tecnológico

El enunciado sugiere React, Express y MySQL, pero acepta variantes justificadas. Mantuve Laravel porque es el proyecto que ya tenía funcionando, y los conceptos de SQA son los mismos: cambia la herramienta que cumple cada papel.

Capa	Herramienta que usé	Equivalente sugerido
Aplicación	Laravel 13, PHP 8.4, Blade y MySQL 8	React, Express y MySQL
Repositorio y CI/CD	GitHub y GitHub Actions	GitHub Actions
Pruebas	PHPUnit 12 (SQLite en memoria; MySQL 8 para reportes)	Jest, Supertest
Cobertura	PCOV, reporte Clover y un script de umbral	Istanbul, nyc
Estilo	Laravel Pint	ESLint, Prettier
Análisis estático	Larastan (PHPStan nivel 5) y SonarQube Cloud	SonarCloud
Documentación	Markdown en la carpeta docs	Markdown

3. Arquitectura del pipeline

El workflow está en .github/workflows/ci.yml y corre en cada push y en cada pull request. Build prepara las dependencias. Después corren en paralelo las pruebas con SQLite y las de reportes con MySQL. Si ambas pasan, Analyze ejecuta Pint, PHPStan y el escaneo de Sonar, y al final Report arma el resumen.



Cada pull request hacia main debe pasar los cinco checks. Si uno falla, GitHub bloquea el merge.
Quality gates: cobertura de líneas ≥ 60 %, Pint sin fallas, PHPStan nivel 5 sin errores, Quality Gate de Sonar.

3.1 Quality gates

Gate	Umbral	Qué pasa si falla
Pruebas PHPUnit	Todas aprobadas	Falla el job Test y se omite Analyze
Cobertura de líneas	≥ 60 %	Falla el paso Quality gate del job Test
Estilo (Pint)	Sin archivos con fallas	Falla el job Analyze
PHPStan nivel 5	0 errores	Falla el job Analyze
SonarQube Cloud	Quality Gate Passed	Falla el job Analyze
Protección de main	5 checks y PR obligatorios	GitHub bloquea el merge

4. Métricas de línea base (antes)

Medí el proyecto original antes de tocar nada. Los datos salen de ejecutar PHPUnit, Pint y PHPStan sobre el código inicial y del primer análisis automático de SonarQube Cloud.

Métrica	Valor inicial
Pipeline CI/CD	No existía
Pruebas totales	8 (22 aserciones), todas de ejemplo o de reglas de negocio
Pruebas unitarias propias	0
Cobertura de líneas	No medida; no había herramienta configurada
Estilo (Pint)	2 archivos con fallas, de 53
PHPStan nivel 5	19 errores
Sonar, primer análisis (3.2k líneas con vistas Blade)	Security 12, Reliability 82, Maintainability 1, duplicación 0.9 %
Protección de la rama main	No

Los 19 errores de PHPStan eran sobre todo tipos de las relaciones Eloquent, bloques catch que PHPStan consideraba inalcanzables y una llamada a abort_unless con un objeto. La lista completa quedó guardada en docs/evidencias/logs/phpstan-errores.txt.

5. Métricas finales (después)

Métrica	Antes	Después
Pruebas totales	8	58 (135 aserciones; 2 omitidas en SQLite y ejecutadas en MySQL)
Pruebas unitarias propias	0	15
Pruebas de regresión dedicadas	0	6
Cobertura de líneas	No medida	60.13 % en CI (273/454) y 61.3 % en SonarQube Cloud
Estilo (Pint)	2 archivos	0; PASS en 61 archivos
PHPStan nivel 5	19 errores	0
Quality Gate de Sonar	No existía	Passed
Rama main protegida	No	Sí: PR y cinco checks obligatorios

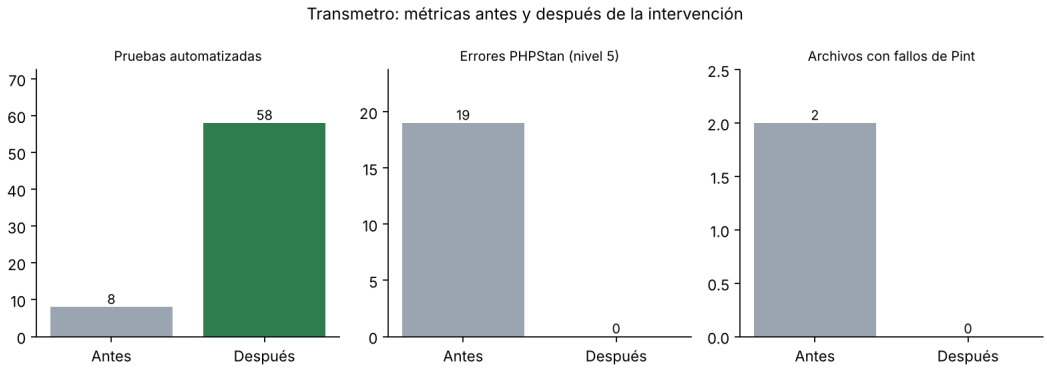


Figura 1. Pruebas, errores de PHPStan y fallas de Pint, antes y después.

Aclaración sobre Sonar: el primer análisis incluía las vistas Blade y marcó Security 12 y Reliability 82. El análisis del pipeline excluye resources/views por configuración y reporta 0 issues. Esa baja se debe a la exclusión y no a correcciones, y los issues de las vistas siguen sin revisar. Por eso no la cuento como mejora. Las correcciones reales son las de Pint, Larastan, las pruebas y el pipeline.

6. Capturas del pipeline funcionando

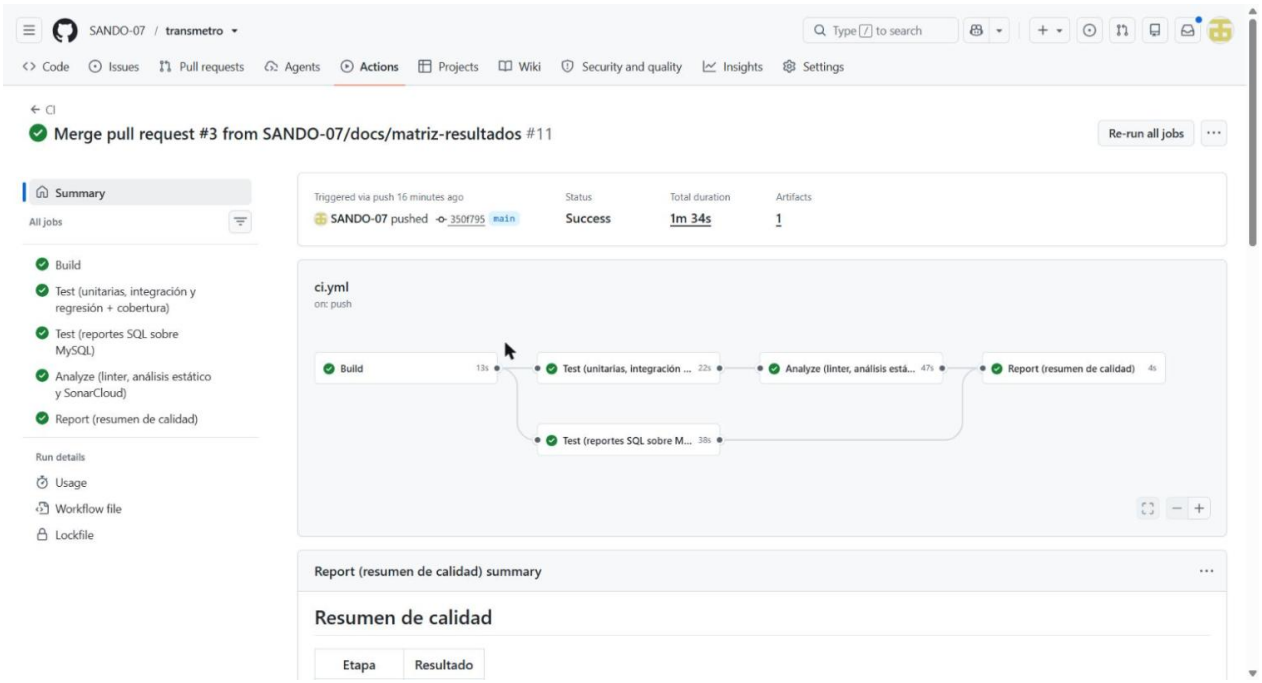


Figura 2. Etapas del pipeline en GitHub Actions: Build, Test, Test MySQL, Analyze y Report.

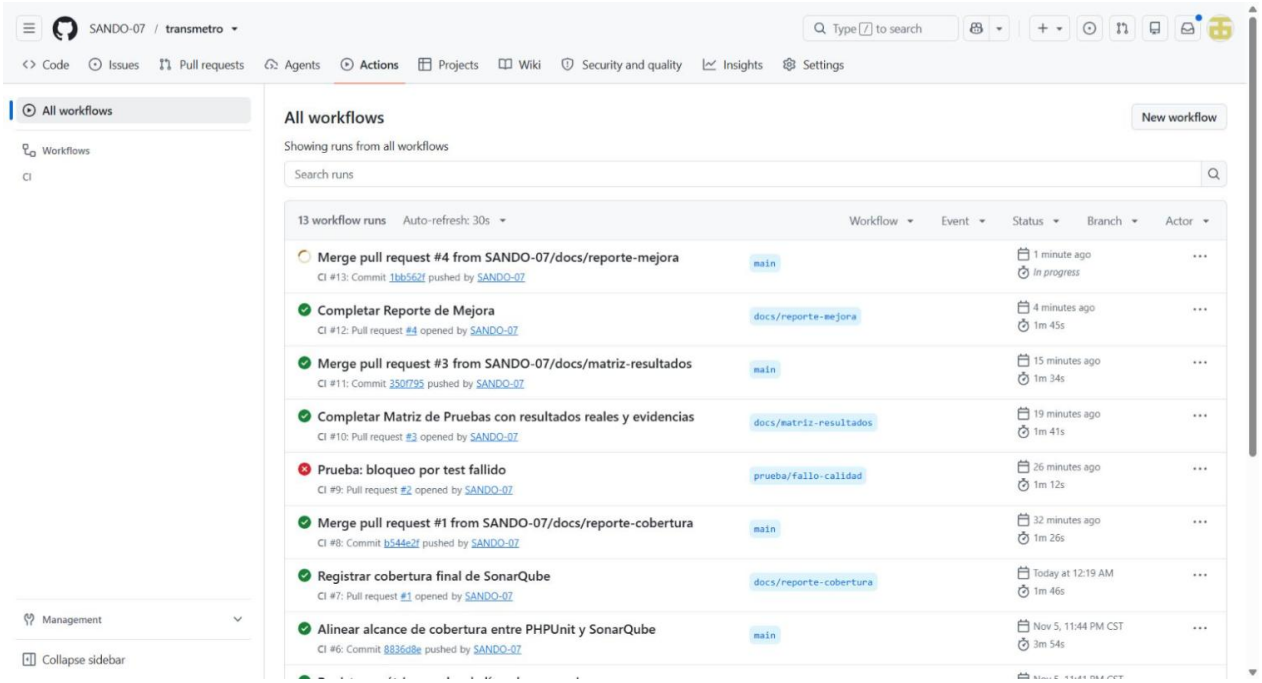


Figura 3. Historial de ejecuciones, con el rojo intencional del pull request de prueba.

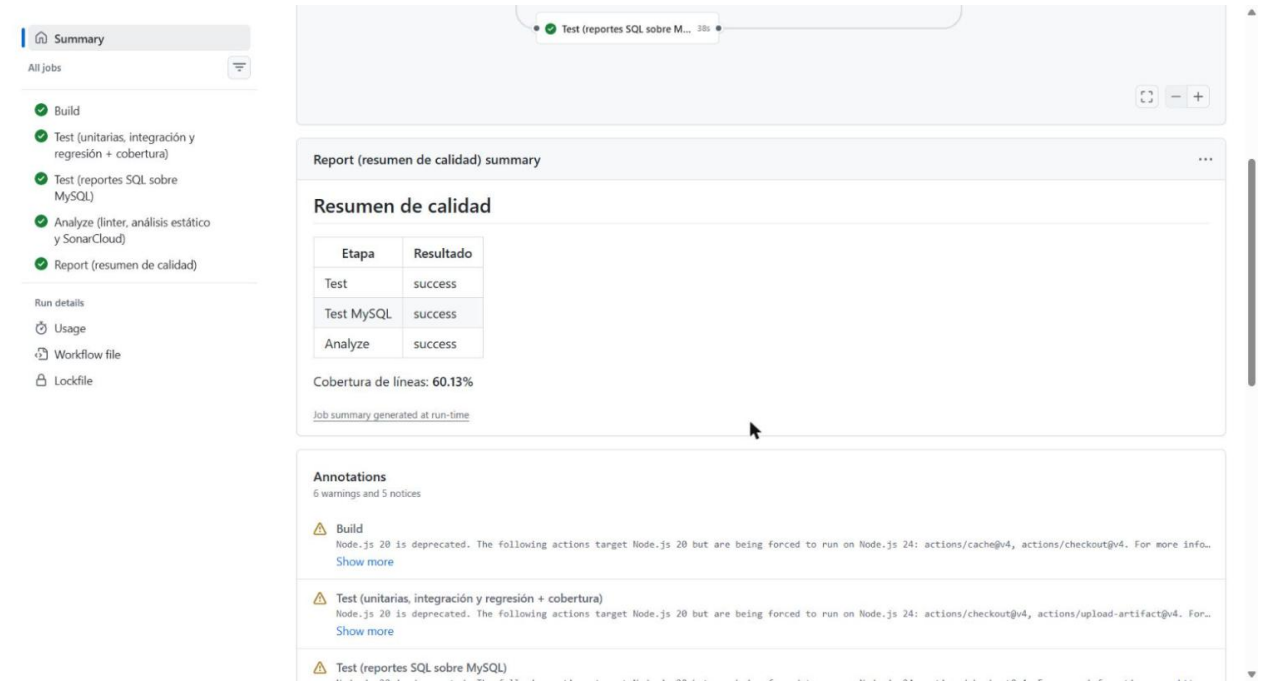


Figura 4. Resumen de calidad generado por el job Report, con la cobertura de líneas.

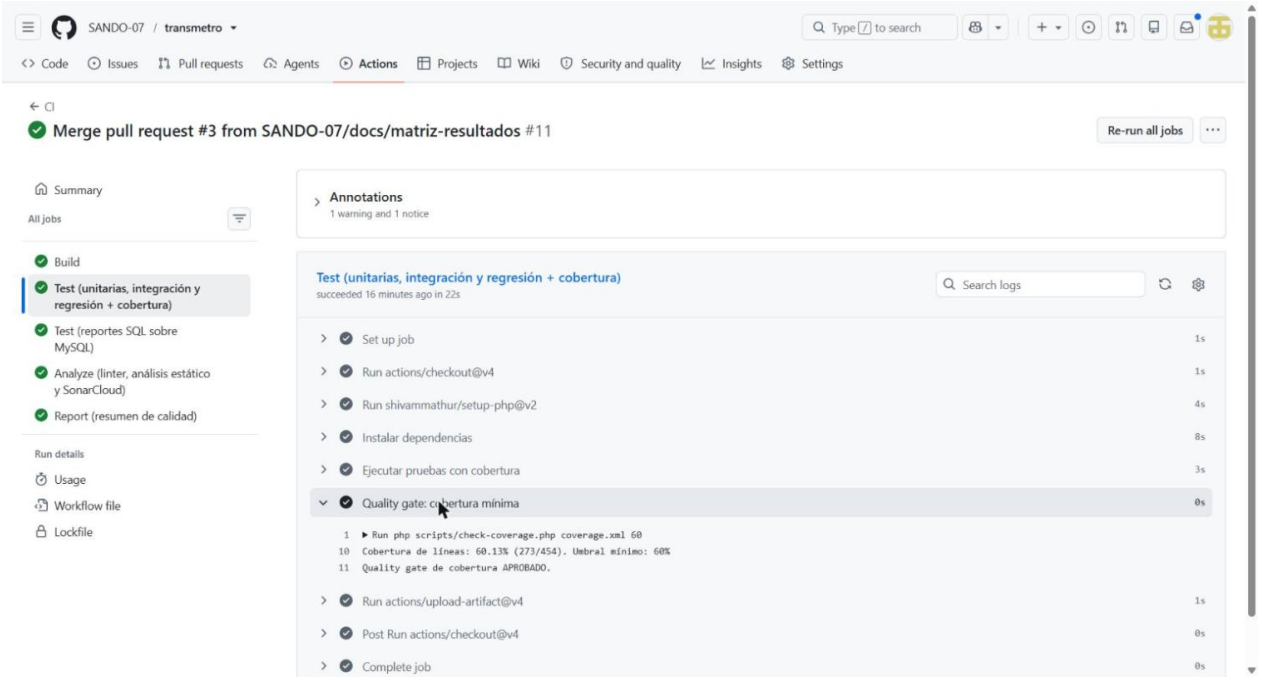


Figura 5. Quality gate de cobertura: 273 de 454 líneas, umbral 60 %, aprobado.

6.1 Prueba negativa del quality gate

Para comprobar que el pipeline de verdad bloquea código defectuoso, abrí el pull request número 2 con un test roto a propósito: cambié el valor esperado de una aserción en ReglasServicioTest. El job Test falló, Analyze se omitió y GitHub deshabilitó el botón de merge. Cerré el pull request sin mergear y borré la rama.

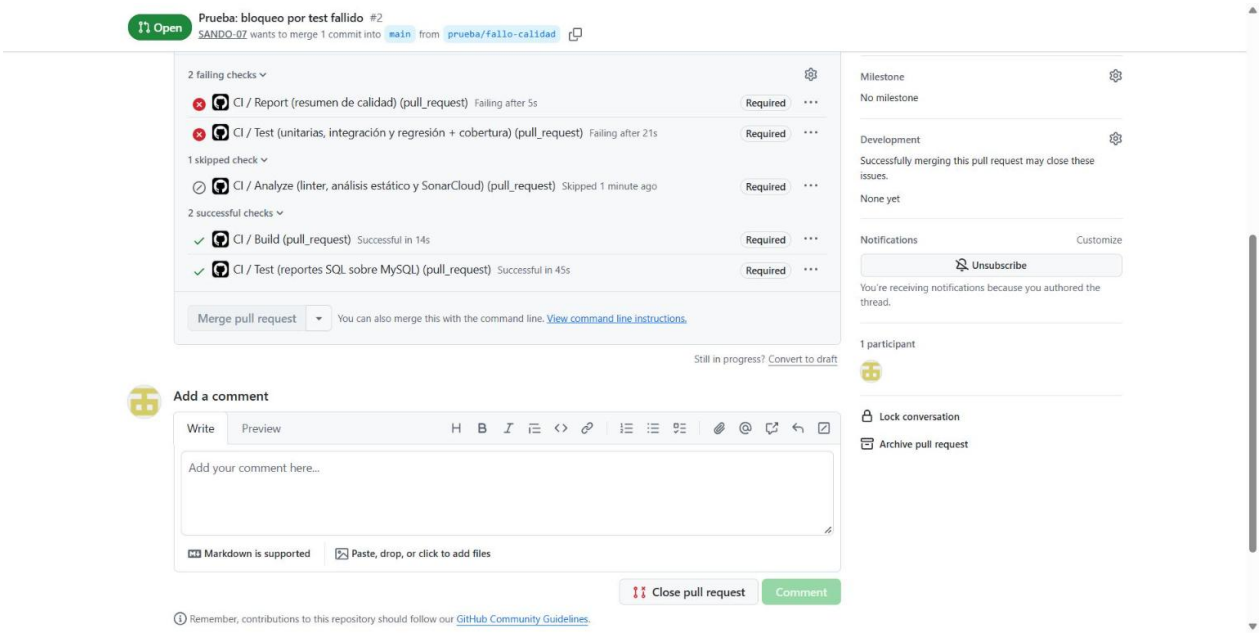


Figura 6. Pull request #2: checks fallidos y merge bloqueado.

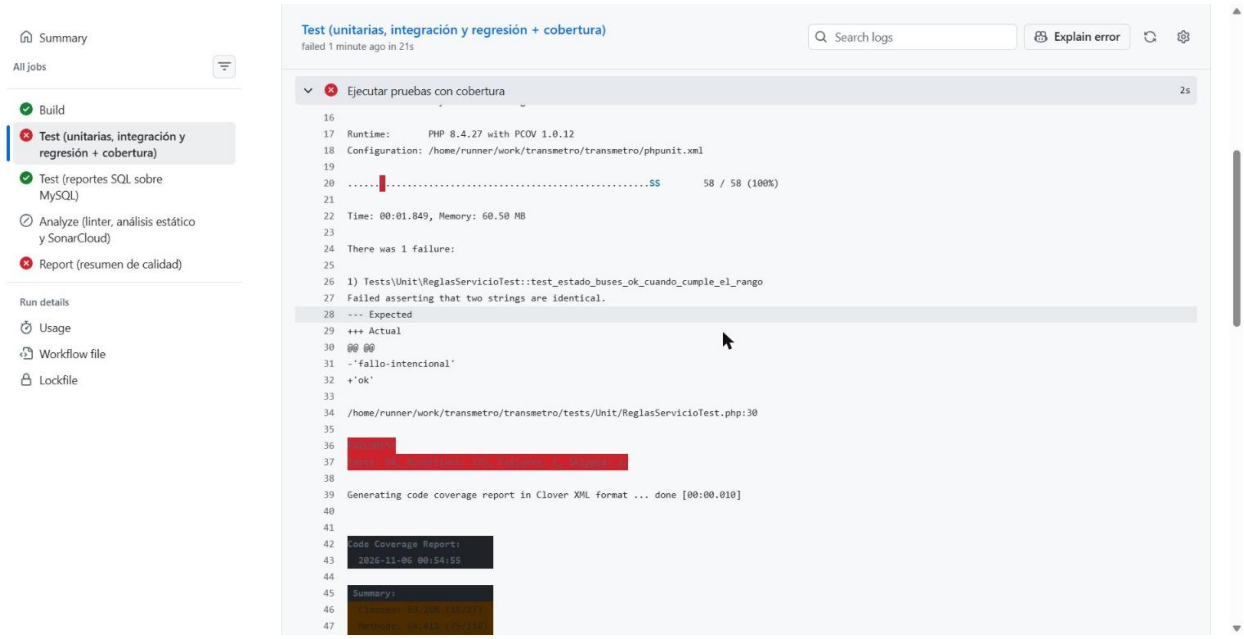


Figura 7. Log del job Test con la aserción que falló.

7. Capturas del dashboard de calidad

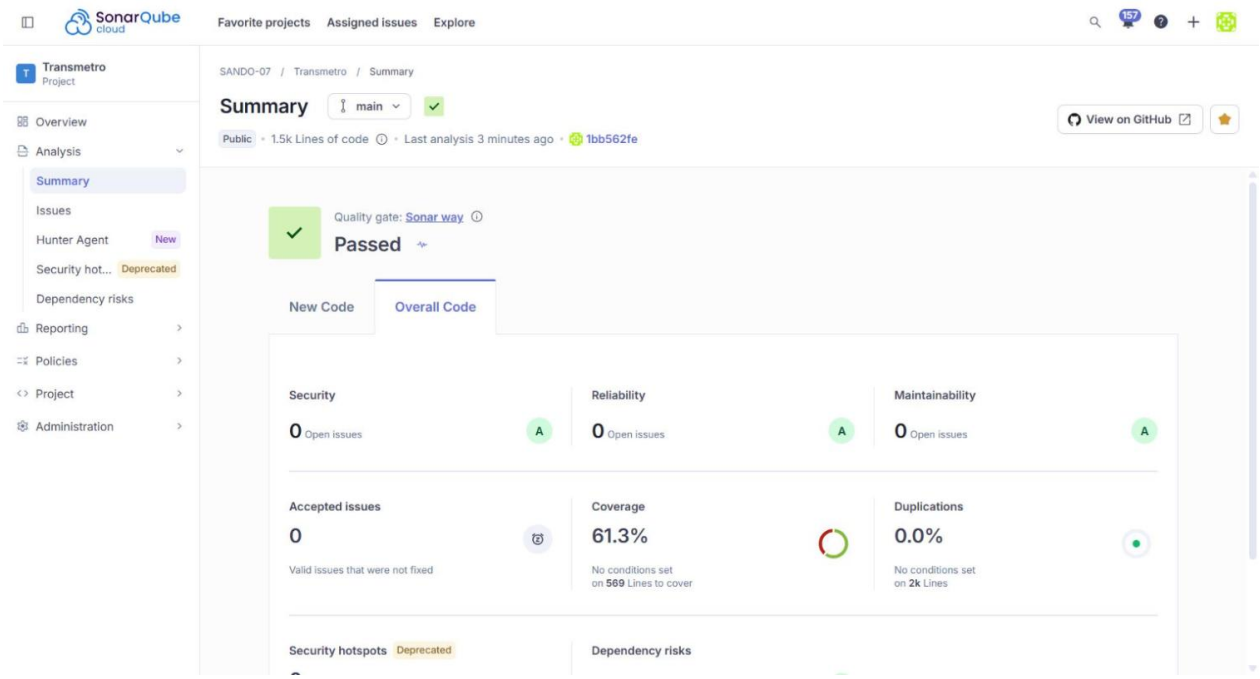


Figura 8. SonarQube Cloud: Quality Gate Passed, 0 issues, cobertura 61.3 % y duplicación 0.0 %.

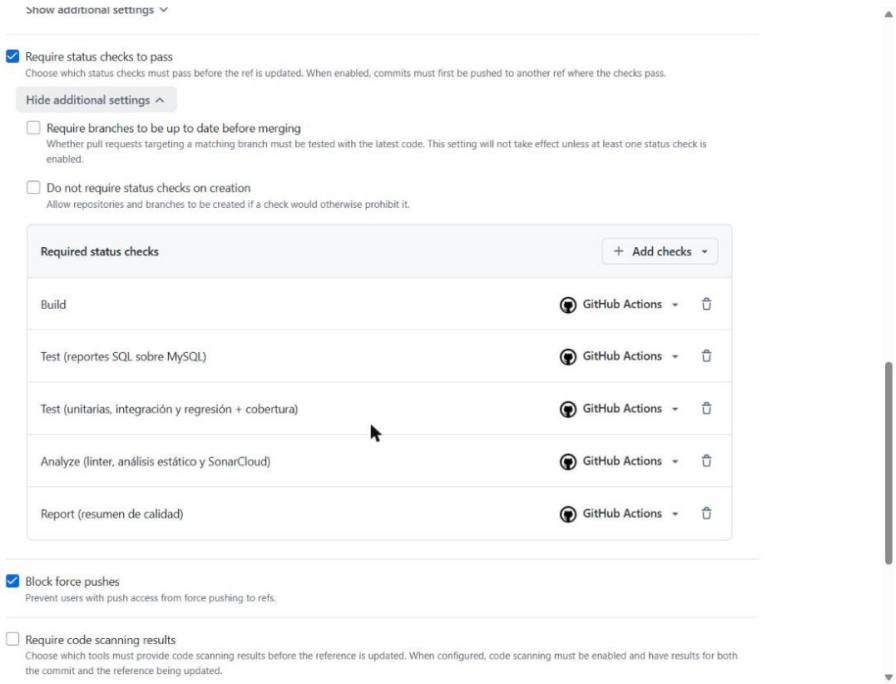


Figura 9. Regla de protección de main: pull request obligatorio y los cinco checks requeridos.

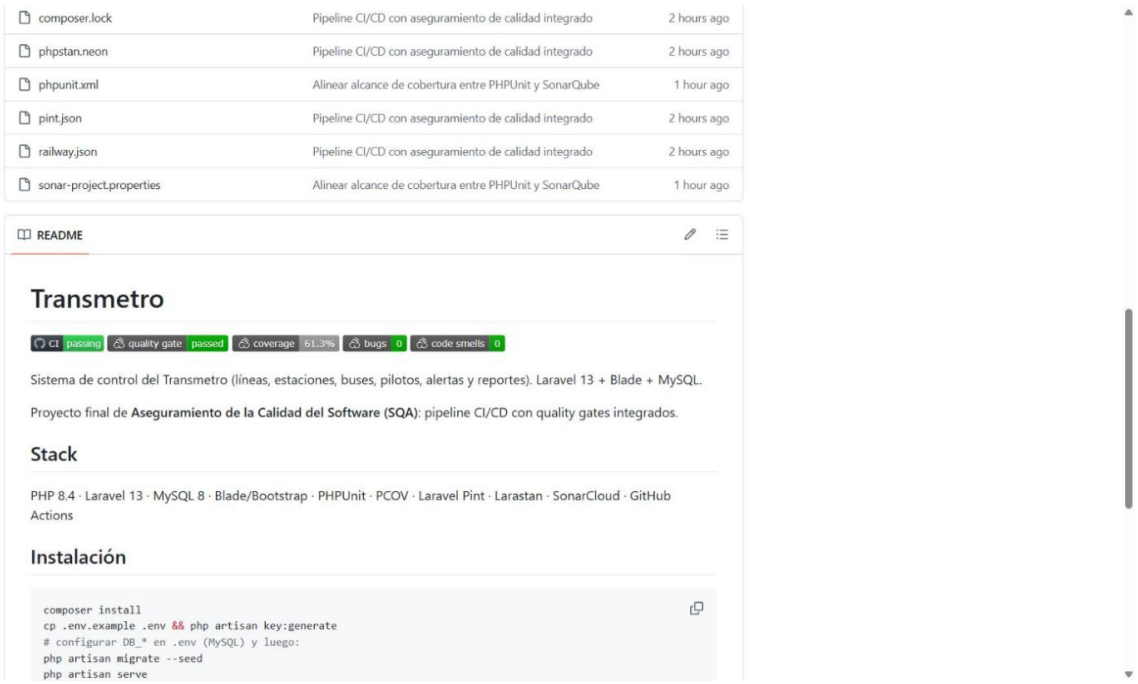


Figura 10. README del repositorio con los badges de CI, quality gate, cobertura, bugs y code smells.

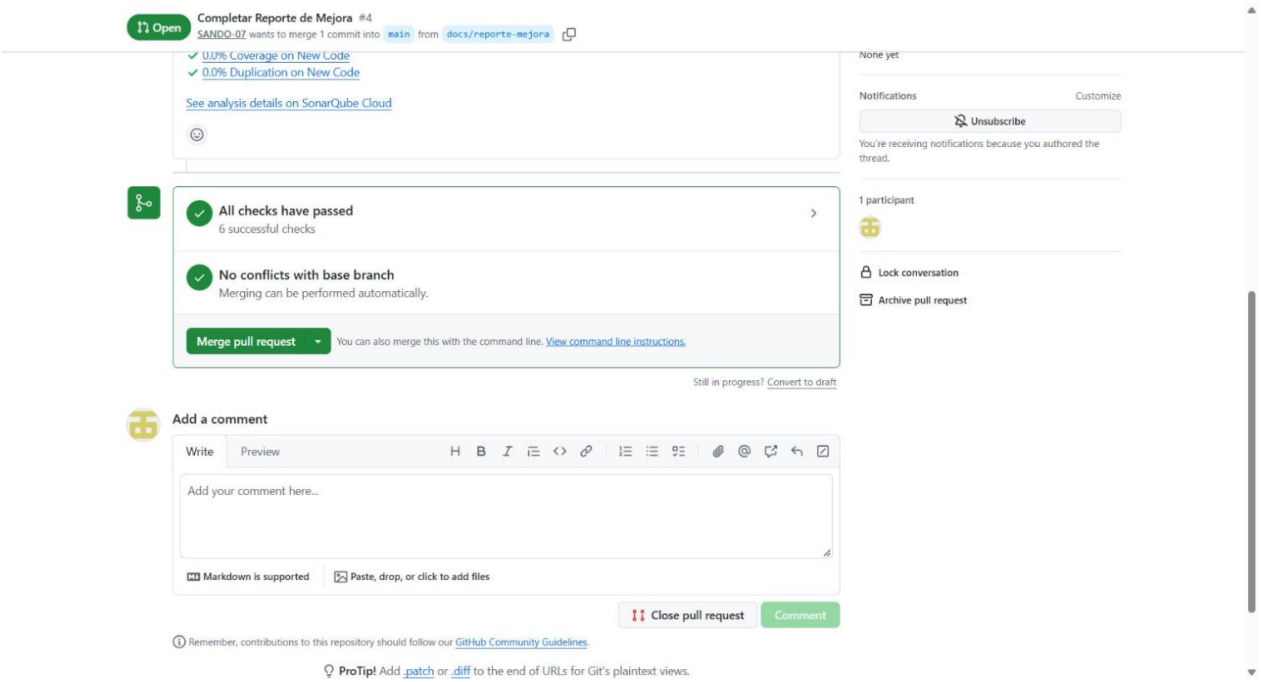


Figura 11. Pull request con los seis checks aprobados y el merge habilitado.

8. Conclusiones y lecciones aprendidas

El proyecto pasó de no tener ninguna verificación automática a un flujo donde nada entra a main sin pasar pruebas, estilo, análisis estático, cobertura mínima y el Quality Gate de Sonar. Las pruebas subieron de 8 a 58, la cobertura quedó sobre el 60 % pedido y los 19 errores de PHPStan y las fallas de Pint quedaron en cero.

8.1 Dificultades y cómo las resolví

1. Las pruebas de reportes necesitan MySQL y en CI no se ejecutaban. PHPUnit 12 ya no lee la anotación de grupo; usé el atributo #[Group('mysql')].
2. Sonar daba 45 % de cobertura y PHPUnit más. Medían alcances distintos; alineé phpunit.xml y sonar-project.properties y Sonar subió a 61.3 %.
3. La regla de protección de main bloqueaba todo aunque los checks estaban en verde. Los nombres de los checks requeridos no coincidían con los reales, por una tilde dañada y nombres cortos. Los reemplacé por los nombres exactos.
4. En Windows tuve problemas de codificación en ci.yml, de PATH y de la extensión sqlite de PHP. Los corregí a mano y quedaron documentados en el Manual Técnico.
5. Las vistas Blade inflaron el primer análisis de Sonar. Las excluí del análisis y lo dejé dicho en el reporte para no presentarlo como mejora.

8.2 Lo que aprendí

Un quality gate sirve solo si se ha visto fallar. Por eso incluí la prueba del test roto. También aprendí que una métrica solo se puede comparar si se mide con el mismo alcance, y que es mejor explicar una diferencia que dejar que parezca una mejora.

9. Referencias

1. Repositorio del proyecto: https://github.com/SANDO-07/transmetro
2. Panel de SonarQube Cloud: https://sonarcloud.io/project/overview?id=SANDO-07_transmetro
3. Documentación de PHPUnit: https://phpunit.de
4. Documentación de Laravel Pint: https://laravel.com/docs/pint
5. Larastan: https://github.com/larastan/larastan
6. GitHub Actions: https://docs.github.com/actions
7. Reglas de ramas (rulesets) de GitHub: https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets
8. Documentos del proyecto, en la carpeta docs del repositorio: SQA-Plan.md, Matriz-Pruebas.md, Linea-Base-Calidad.md, Reporte-Mejora.md y Manual-Tecnico.md.